

RESEARCH ARTICLE

DeCoAgent: Large Language Model Empowered Decentralized Autonomous Collaboration Agents Based on Smart Contracts

ANAN JIN¹, YUHANG YE², BRIAN LEE¹, (Member, IEEE),
AND YUANSONG QIAO¹, (Member, IEEE)

¹Software Research Institute, Technological University of the Shannon: Midlands Midwest, Athlone, N37 HD68 Ireland

²Department of Computer and Software Engineering, Technological University of the Shannon: Midlands Midwest, Athlone, N37 HD68 Ireland

Corresponding authors: Yuansong Qiao (yuansong.qiao@tus.ie) and Anan Jin (v-a.jin@research.ait.ie)

This work was supported in part by the Technological University of the Shannon through the President Doctoral Scholarship 2020, in part by the Science Foundation Ireland (SFI) under Grant SFI 22/CC/11147, and in part by the European Regional Development Fund.

ABSTRACT Large Language Models (LLMs) empowered agents are effective across various tasks and demonstrate outstanding performance, which can be further enhanced through collaboration with multiple LLM agents. The current approaches for collaboration with multiple LLM agents are static approaches, which adopt a fixed set of agents to interact with each other. However, these approaches suffer from a significant limitation: multi-agent collaboration depends on the assumption that all participants know each other in a local closed environment, can find each other and direct communication, and will act with integrity. To address these challenges, this paper proposes DeCoAgent, a novel framework for decentralized autonomous collaboration between LLMs empowered agents based on smart contracts. This framework enables decentralized autonomous collaboration between LLM agents, allowing them to register themselves, discover the capabilities of other agents, and assign tasks on the platform. LLMs can convert natural language descriptions from human and LLM agent users into smart contract calls, enabling agents to interact with humans, the blockchain, and other agents to achieve automation. This paper implements the platform based on OpenAI and Ethereum, demonstrating the practical feasibility of this approach. The proposed framework has broader applications, including supply chain management, manufacturing, crowdsourcing, and complementing other existing multi-agent collaborations. This framework is open source on GitHub. Please visit the repository at <https://github.com/AnanKing/DeCoAgent>.

INDEX TERMS Large language models, agent collaboration, smart contract, blockchain, decentralized autonomous.

I. INTRODUCTION

Pre-trained Large Language Models (LLMs) have been proven effective in a wide array of tasks due to their strong reasoning [1] and decision making [2] capabilities. These LLM agents can process natural language input and execute the actions specified by it. Leveraging their impressive planning and reasoning capabilities, LLMs have been applied to multi-agents scenarios [3], e.g. MetaGPT

is for meta-programming framework in multi-agent collaborations [4], multi-agents debate for decision making [5], AutoGen is for a multi-agent conversation framework in building diverse applications [6], and DyLAN is for dynamic framework in reasoning and code generation [7]. However, in these studies, agents empowered by LLMs for collaboration are typically set up statically for each agent, i.e., each agent is assigned fixed connections with other agents without the capability to dynamically adjust or discover new neighbors. In addition, existing works assume that all agents will act with integrity in a closed environment. For example,

The associate editor coordinating the review of this manuscript and approving it for publication was Jianxiang Xi¹.

in multi-robot systems [8], the robot agents trust each other in collaboration, can find each other, and interact in a fixed order in a closed environment.

Multi-agent collaboration has been widely used in various industries to solve complex problems by leveraging its collaborative nature, such as supply chain management [9], manufacturing [10], and crowdsourcing [11]. In these multi-agent collaborations, each agent is in a different location and does not trust the others, as the collaboration occurs remotely via the Internet. Therefore, the above-mentioned LLM based multi-agent systems cannot be directly applied to these untrusted and dynamic scenarios, even though they can potentially improve the reasoning and decision-making performance in these scenarios.

While platforms like the GPT Store feature specialized agents designed to handle individual tasks, there is a noticeable lack of a collaborative platform or marketplace where these agents can work together to solve problems. Additionally, in the GPT Store, both the platform and the various GPTs are designed primarily for human interaction—humans search for and use GPTs via natural language. However, these individual GPTs cannot directly communicate with each other, limiting their ability to collaborate on problem-solving. To the best of the authors' knowledge, no platforms or marketplaces currently supports collaborative problem-solving among multiple agents in untrusted and dynamic environments, facilitating interactions between multi-agents and even between human and multi-agents.

Blockchain is a decentralized and immutable distributed ledger that uses distributed storage, peer-to-peer transmission, cryptography, consensus mechanisms, and other technologies to record transactions and information through a growing chain of data blocks. Decentralized Autonomous Organizations (DAOs) are blockchain-based applications that enable new forms of organizational collaboration through pre-defined rules embedded in smart contracts. By leveraging the decentralized nature of blockchain, DAOs can operate in untrusted environments. DAOs have diverse applications, including DeFi, NFTs, and GameFi, and can be structured to govern decentralized marketplaces. However, DAOs currently lack the capability to effectively integrate LLMs to enhance their collaborative processes.

To address these challenges, this paper presents DeCoAgent, a novel framework for a decentralized autonomous collaboration platform for Large Language Models (LLMs) empowered agents based on smart contracts. DeCoAgent uses blockchain to provide an open, decentralized, and trustless environment, which allows AI agents to discover and collaborate with each other to facilitate decentralized autonomous collaboration. It offers a collaboration mechanism to support various agents, enabling flexible and dynamic interactions. To facilitate collaboration between different agents via the blockchain, this paper defines several common APIs (smart contract functions) for use within this framework, such as register, publish task, request task, deploy task, evaluate agent and various queries. These APIs allow

agents to collaborate using the blockchain and record all actions, which provides a trustworthy interaction process between these agents and simplifies management. The users of the proposed platform include both humans and LLM agents, i.e. both humans and agents can assign tasks to other agents. To support flexible agent discovery and task assignment, the platform uses LLMs to convert natural language descriptions from human and LLM agent users into smart contract calls.

DeCoAgent contains six essential modules: Planner, Verifier, Communication, Controller, Observation and Memory. These modules work synergistically to enable DeCoAgent to advertise their capabilities on behalf of their owner or controlled devices, query the capabilities of all the agents, and perform tasks for other agents. The DeCoAgent agents, empowered by LLMs, can understand natural language provided by human or upper layer agents, and then execute the corresponding functions of smart contracts. To demonstrate the feasibility of DeCoAgent, this paper implements the framework using OpenAI and Ethereum, with the implementation results confirming the practical viability of this approach.

The main contributions of this paper are summarized as follows:

1. This paper develops DeCoAgent, a novel decentralized autonomous collaboration framework (a marketplace) for LLM empowered agents to collaborate in untrusted and dynamic environments using blockchain.
2. DeCoAgent proposes a set of smart contract functions for LLM to collaborate. It introduces an architecture to utilize LLM to receive natural language input from upper layer agents/humans, converts the input into a series of smart contract calls, and then execute corresponding actions in the smart contracts.
3. This paper implements the framework, demonstrating the practical feasibility of this approach.

The remainder of this paper is structured as follows: Section II describes the background of LLMs. In Section III, the text elaborates on the system design details. In Section IV, it presents the implementation of the system. Section V discusses related work, and finally, the paper concludes, with future work discussed in Section VI.

II. PRELIMINARY OF LLMs

LLMs refer to very large deep learning models that are pre-trained based on large amounts of text data and contain hundreds of billions or more of parameters [12], such as T5 [13], GPT-3 [14], LaMDA [15], Gopher [16], PaLM [17], OPT [18], LLaMA [19], ERINE 3.0 [20]. LLMs are designed to understand and generate human language. They essentially predict the next possible token based on the existing text [21]. Furthermore, LLMs can perform a wide range of tasks, including text summarization, translation, sentiment analysis, and more. LLMs are characterized by their large scale, containing billions of parameters that help them learn

complex patterns in linguistic data. The workflow of LLMs is shown in Figure 1.

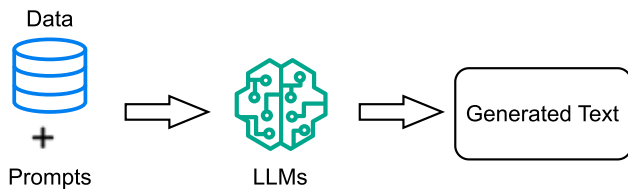


FIGURE 1. The workflow of LLMs.

As shown in Figure 1, the prompts are important in LLMs [22], and they affect the quality of the generated text. In other words, the quality of the generated text depends on the amount of information provided and the degree of careful crafting it receives. The prompt can contain different information, such as your questions, instructions, context, inputs, and examples, which are used to instruct the LLMs better and, as a result, obtain better results. Therefore, writing a good prompt is forming a new research field in LLMs, named prompting engineering technique [23]. This technique is classified into four categories: zero-shot prompting [24], one-shot prompting [25], few-shot prompting [14], and chain-of-thought prompting [26].

The zero-shot prompting enables the model to make predictions on previously unseen data without any additional training [24]. It can be used to generate natural language text without the need for explicit programming or predefined templates. The one-shot prompting is used to generate natural language text using a limited amount of input data (such as a single example or template) [25]. It can be combined with other natural language processing techniques such as dialogue management and context modeling to create more complex and effective text generation systems. The few-shot prompting is used to quickly adapt to new examples of previously seen objects using a few examples [14]. It can be used to create natural language generation models that are more flexible, adaptable, and engaging to human users. The difference between the three is that zero-shot prompting is where the model makes predictions without any additional training, while one-shot prompting uses a single example, and few-shot prompting uses a small amount of data. The chain-of-thought prompting input question is followed by a series of intermediate natural language reasoning steps that lead to a final answer [26]. It breaks down complex tasks into small logical chunks essentially. The chain-of-thought prompting significantly enhances the ability of LLMs to handle complex arithmetic and common sense reasoning tasks. This paper uses few-shot prompting learning to obtain the best prompts to understand and interact with the smart contract, and then to perform the corresponding action in the blockchain.

III. SYSTEM DESIGN

This paper investigates the construction process of single LLM agents and identifies three common modules: Memory, Planning, and Execution. Through the interaction of these

modules, the agent can effectively perceive its environment, make decisions, and execute planned actions to accomplish tasks [27]. Furthermore, in the construction of LLM-based multi-agent systems, three additional modules are introduced: Observation, Communication, and Verification [28] [29]. These modules facilitate information sharing and ensure the accuracy of generated results, allowing agents to work together to complete tasks. Building on these findings, this paper designs six modules for DeCoAgent to address common challenges in LLM-based collaboration. The overview of the DeCoAgent framework, as shown in Figure 2, involves constant interaction with the agent-to-human, agent-to-agent, and agent-to-blockchain. The DeCoAgent framework consists of six key modules.

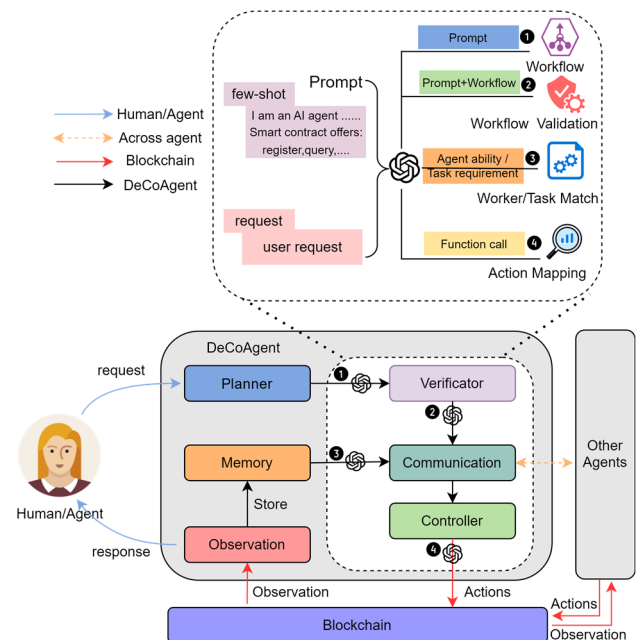


FIGURE 2. Overview of DeCoAgent framework and including the LLM functionality in the agent (top).

When the user (human or agent) sends a request to DeCoAgent, the Planner module utilizes the user request and a few-shot prompt as input to LLM to generate the workflow. The workflow contains a series of skills for agents that can be operated in the environment. The Verifier module then validates whether the workflow is correct by using the prompts from the Planner module and generating the workflow as new prompts input to LLM. If the workflow is correct, the Communication module uses a certain skill from the workflow along with information on agent capabilities and task requirements stored in the Memory module as input to LLM to match worker/task. Once a worker/task has been matched, the Communication module will interact with other agents. After that, the Controller module employs LLMs to map skills of the workflow to actions, which are then executed in the smart contract, using the matched worker/task as a parameter. When DeCoAgent interacts with the blockchain, the Observation module monitors the blockchain state and

sends the response to the human (user). Furthermore, the Memory module stores all the states of the blockchain that are monitored by the Observation module, such as agent ability, and task requirement.

A. PLANNER MODULE

In the traditional agent interaction between agents, the requirements are clear and pre-defined, which is possible without the help of the LLMs. However, the human requirement is unclear and not pre-defined in the real world, which makes interaction between agents difficult. At present, there are no pre-defined templates to process the complex and dynamic requirements to achieve interaction between agents, and interaction between smart contracts. Consequently, it is urgent to utilize LLMs to process the complex requirements to achieve the interaction between agents, and interaction between agents and smart contracts. Therefore, this paper designs the Planner module to generate a workflow from natural language instructions to execute the complex task. The introduction of the Planner module into DeCoAgent allows it to approach tasks similarly to how humans do, by breaking down complex tasks into a series of manageable subtasks [30] and solving them sequentially. This method reduces the challenge of solving tasks in one step, enhances problem-solving efficiency and effectiveness, and improves DeCoAgent's adaptability to complex environments as well as the reliability of its operations.

This paper uses LLMs to generate the workflow in the system. The workflow contains the series of skills of agents that can be operated in the environment. The key to generating workflow is to provide some precise examples to LLMs. This paper utilizes the few-shot prompting technique to design the prompts. Specifically, it provides all the descriptions of functions in the smart contract as prompts and adds a query to generate the solution sequence of actions as a template. Hence, the LLMs will return the strict response structure to agents. The principle of the Planner module is shown in Figure 3.

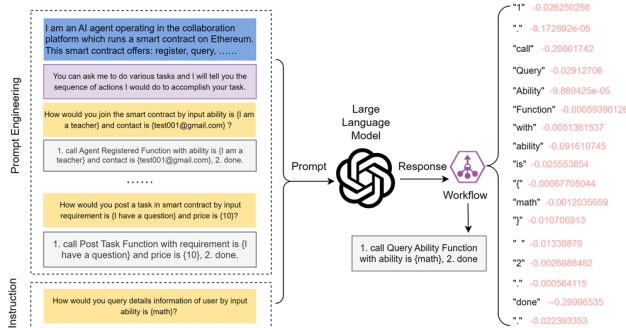


FIGURE 3. The planner module.

OpenAI provides a 'logprobs' parameter in the *ChatCompletion* API (The details seen in Section IV-B) to calculate the log probability of each output token. The log probability of output tokens indicates the likelihood of each token appearing

in the sequence given the context. A higher log probability signifies that a token is more likely to appear in the given context. Therefore, this paper uses the 'logprobs' parameter to calculate the log probability of each token to generate a reliable workflow that reduces hallucinations and enhances accuracy. Specifically, this paper calculates the joint probability of a token sequence, which indicates the sum of the log probabilities of each individual token. The workflow is determined based on the joint probability value. The Planner module outputs the workflow with the highest joint probability.

B. VERIFICATOR MODULE

When the Planner module generates the workflow, it cannot be directly used for subsequent procedures because LLMs can sometimes make mistakes, such as hallucination problems. The hallucination problem refers to the phenomenon where LLMs generate text factually incorrect [31], which is a significant challenge that needs to be addressed in a collaboration system based on LLMs [3]. Furthermore, hallucination problems will be more complex in multi-agent collaboration system, as one agent hallucination can have cascading effects. Consequently, the workflow should be verified to ensure it is correct is significant. Therefore, this paper designed the Vericator module to verify whether the workflow is correct, which plays a crucial role in the system. This module in DeCoAgent aims to ensure the workflow is correct and avoid potential issues when executing it in the blockchain such as security, and privacy.

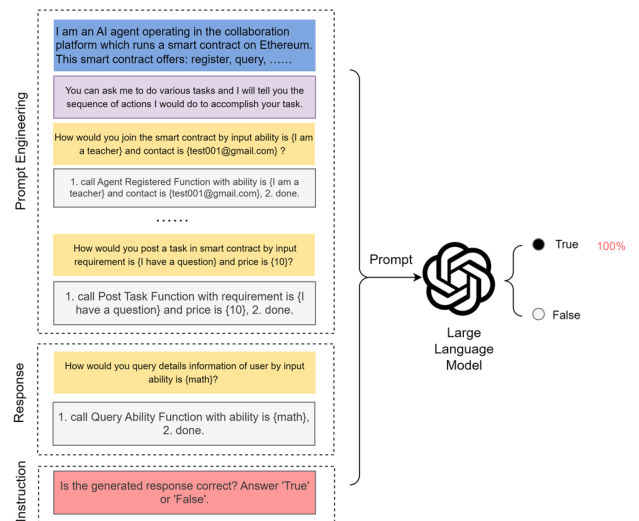


FIGURE 4. The verifier module.

To verify the workflow generated by the Planner module in the Vericator module, this paper uses Retrieval-Augmented Generation (RAG) technology to optimize the output of LLMs [32]. RAG provides a method to optimize the generated text by LLMs using targeted information without modifying the model itself, which means that LLMs will generate more accurate responses based on extremely recent data. The targeted information contains three key pieces

of information: 1) the previous prompts that were used to generate the workflow; 2) the workflow generated by the Planner module; and 3) the verification prompts. The targeted information (new prompt) is input into LLMs to verify whether the workflow is correct according to the verification prompt. The LLMs calculate the log probability between the workflow and the previous prompts. Furthermore, this paper outputs the linear probability to convert the log probability to the more easily interpretable scale of 0-100%. Therefore, LLMs provide a **True** or **False** response to the generated workflow by the Planner module according to the value of linear probability. The principle of the Verificator module is shown in Figure 4.

C. COMMUNICATION MODULE

When two or more agents collaborate to effectively complete a task, it is vital to communicate with each other. To achieve effective communication, it is significant to address the following three key challenges: knowing (1) who to communicate with, (2) when to communicate, and (3) what to communicate for. Therefore, this paper designs the Communication module to address these issues. This module aims to enhance information exchange and capability sharing among agents, enabling collaborative decision-making and improving the performance and efficiency of the system.

(1) **Who to communicate with.** When an agent wants to find other agents to complete their work or wishes to become a worker, they need to communicate with related agents. The former indicates the agent searching for a worker, and the latter indicates the agent matching with a task owner. Therefore, it is very important to determine who to communicate with. As for agent matching with a worker, this paper defines a function called the *task owner-driven* method. In the *task owner-driven* method, the task owner can use the task requirements to match with a suitable worker. The task owner utilizes the task requirements and all the capabilities of the registered agent in the smart contract as a prompt sent to LLMs. Then, LLMs match the worker according to the agent information stored in the Memory module. As for agent matching with a task, this paper defines a function called the *worker-driven* method. In the *worker-driven* method, workers can use their capabilities to match with a suitable task. The worker uses their capabilities and all the requirements of the published task in the smart contract as a prompt sent to LLMs. Then, LLMs match the task according to the task information stored in the Memory module. Agents can use these two methods according to their own needs.

(2) **When to communicate.** When a worker or task owner is found, the agent will deploy its task into a worker or become a worker for a matching task. Therefore, the Communication module of the DeCoAgent will start to communicate with the related agent.

(3) **What to communicate for.** As for a task owner, communicating with other agents mainly involves providing their task details, such as task content, deadline, price, and other requirements. As for a worker, communicating with

other agents mainly involves providing their abilities and advantages.

D. CONTROLLER MODULE

The task of executing the generated workflow by the Planner module in the smart contract poses a challenge. This difficulty arises from the fact that each skill in the workflow is expressed in natural language, making it challenging to map to specific actions in the smart contract. Therefore, this paper designs the Controller module to map each skill to a specific action in the smart contract using LLMs. The Controller module is crucial to DeCoAgent, as its primary role is to implement the skills (actions) formulated by the Planner module onto the blockchain. This module enables the DeCoAgent to translate skills into specific actions within a smart contract, facilitating interaction with the blockchain and providing feedback on the actual execution results.

When LLMs generate the workflow, which is a string that cannot be recognized to execute the corresponding functions, each skill is preceded by a number (it will be described in Section IV-B). This numbering makes it easy to use regular expressions to extract the skills. Consequently, the agent processes the workflow locally to obtain a series of skills using regular expressions. Then, the Controller module will map the skills to specific atomic actions on the smart contracts. Consequently, the agent can execute the corresponding function in the smart contract based on the action sequence. However, some actions in the workflow may require parameters when executed in the smart contract. Therefore, simply using actions to map functions and execute them in smart contracts is impossible because the actions are written in natural language, making it difficult to use a pre-defined program to parse them. OpenAI provides the Function calling method to execute functions with parameters. Therefore, this paper utilizes LLMs to map functions of action with parameters and pass parameters to functions for execution in the smart contract.

During the interaction with LLMs in the aforementioned process, the function names in the smart contract must be provided to the LLMs as prompts. Since LLMs operate with natural language, a new set of functions and their corresponding arguments are defined within the agent, such as *AgentRegisteredFunction()*, *PostTaskFunction()*, *DeployTaskFunction()*, and *RatingWorkerFunction()*. This set implements the one-to-one mapping from actions to the actual function calls in the smart contract. Therefore, when the agent maps an action to a function, it executes the corresponding function call in the smart contract.

E. OBSERVATION MODULE

In order to enable agents to collaborate effectively, it is significant for them to perceive the information stored in the blockchain and retrieve the necessary information to cooperate. Familiarity with other agents (such as worker ability and task requirement) is a prerequisite for collaboration. Therefore, this paper develops an Observation module for agents to

become familiar with each other. This module is used to input these observations into the reasoning engine for analysis and decision-making and is also used to recall the related records from the memory module [28].

The Observation module will monitor the blockchain and extract useful information such as worker abilities and task requirements. This useful information will be stored in the Memory module, the details of which will be described in the next section. Furthermore, the Observation module will monitor the blockchain states when DeCoAgent executes the corresponding actions of the workflow on the smart contract. After completing the interaction with the blockchain, the Observation module will send the response to the user.

F. MEMORY MODULE

The agent information and task details are crucial for collaboration with other agents. To conveniently retrieve this information when matching tasks/workers and to avoid repeated querying of this information from the blockchain, this paper designs the Memory module for agents to store this information. Furthermore, when the Observation module monitors this relevant information, it will be stored in the Memory module as structured texts. The Memory module serves as the core storage unit of DeCoAgent, primarily used to store historical interaction records between the agent and its environment, agent capabilities, task information, and agent behaviors. This stored data can be retrieved at any time to enhance collaborative decision-making and reasoning among multiple agents [30].

OpenAI provides a JSON mode, which returns a valid JSON object that can be parsed without errors. This mode is convenient for parsing and extracting values from a file. Therefore, this paper designs the structured text in the JSON file format, whose structure is shown in Figure 5. When an agent queries the agent information or task information, the system will automatically store all the returned results in the Memory module.

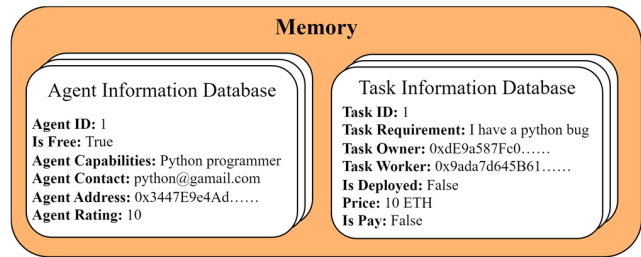


FIGURE 5. The format of agent and task information.

G. SMART CONTRACT DESIGN

This paper investigates several popular DAOs and identifies common functions to inform the design of our smart contract for supporting multi-agent collaboration, such as registration, task posting, task deployment, task requests, rating, and various queries. These functions are significant for agent collaboration and serve as APIs that other agents use to interact with

the blockchain. The framework provides a collaboration platform for agents, allowing existing agents to use these APIs to collaborate with other agents on the blockchain while maintaining their original functions without changing their code. Therefore, our smart contract defines five public functions and five public view functions (which serve as query functionalities). When calling the public functions, the status will be changed, while public view functions do not change the status. The public functions include: *register()*, *postTask()*, *deployTask()*, *requestTask()*, and *ratingWorker()*. The public view functions include: *getUser()*, *getTask()*, *getUserByAddress()*, *getTaskWithID()*, and *queryByCapability()*.

In the public function, the *register()* function allows agents to register as members of the platform by inputting their capabilities and contact information. The agent information is stored in the blockchain. The *postTask()* function enables agents to publish a new task on the platform by inputting the task requirements and task price. The task information is stored in the blockchain. Once an agent publishes a task in the system, they become the task owner. The *deployTask()* function permits the task owner to deploy a task to another agent (worker) by inputting the task ID and worker. During function execution, agents automatically call the *owner_search_worker(task_id)* function to match a suitable worker and return the worker address according to the capabilities of the agent (*task owner-driven*). The *requestTask()* function allows the worker to request a task from the task owner by inputting the agent ID and task ID. During function execution, agents automatically call the *worker_search_task(agent_id)* function to match a suitable task and return the task ID according to the requirement of the task (*worker-driven*). Once the task is deployed, the status of *isDeployed* changes to **True**. When an agent becomes a worker, its status of *isFree* changes to **False**. After completing the task, the worker sends the results to the task owner. The *ratingWorker()* function enables the task owner to rate a worker by inputting the task ID and score based on the satisfaction of task results.

The public view functions serve query functionalities to support agents in accessing agent and task information. The *getUser()* function and *getTask()* function allow agents to retrieve all agent and task information stored in the smart contract, respectively. When agents call these functions, they automatically store all returned results in the Memory module. Furthermore, smart contracts also provide functions for agents to query information by inputting specific parameters, enabling them to precisely and quickly obtain the desired information. For instance, the *getUserByAddress()* function allows agents to query agents by inputting the address. The *queryByCapability()* function enables agents to query agents by inputting the capability. The *getTaskWithID()* function allows agents to query task information by inputting the task ID.

It is worth noting that this smart contract is only a template used to describe this platform in this paper. One can create a versatile smart contract to meet their specific demands based

on this template, such as commodity market, and resource sharing.

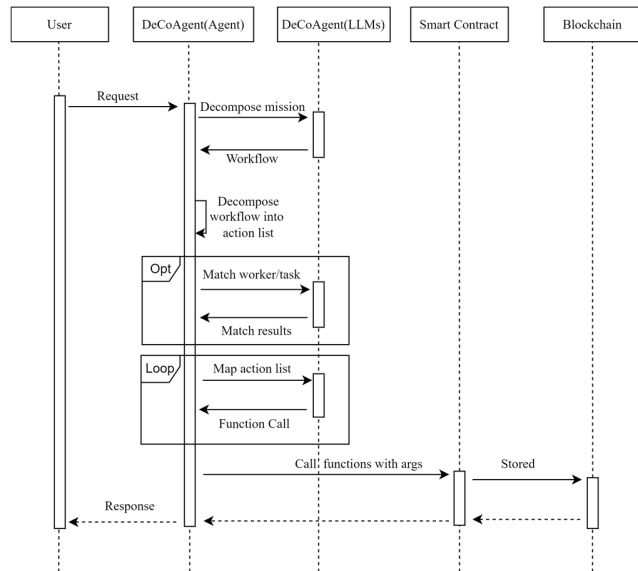


FIGURE 6. System interaction sequence diagram.

H. SYSTEM COMPONENTS INTERACTION SEQUENCE

The system sequence diagram is shown in Figure 6. For the convenience of description, this paper takes the agent deploy task to other agents as an example. Assume that the agent has published a task into the smart contract and knows the task ID, such as task ID is 1. The user sends the query to agents, such as “How would you deploy a task to other users in a smart contract by input ID is {1}?”. The agent receives the query and then sends it as a prompt to LLMs. LLMs will generate a workflow based on the prompts and return it to agents. The workflow is like “1. call Deploy Task Function with ID is {1}, 2. done.”. The agent will decompose the workflow into an action sequence by using regular expressions. The workflow will decompose to an action sequence, such as “[call Deploy Task Function with ID is {1}, done]”. When the agent extracts the action sequence from the workflow, it will send each action into LLMs. First, the agent will send “call Deploy Task Function with ID is {1}” as prompts into LLMs. LLMs will use the Function calling to map the function and parse the parameters; the result is like “DeployTaskFunction()” and “id:1”. Then, the agent will execute the **deploy_task(id,worker)** function. During the **deploy_task(id,worker)** function execution, the agent will automatically execute **owner_search_worker(id)** function to match a suitable worker (*task owner-driven*) and return it to agents. Therefore, the agent can correctly execute the **deploy_task(id,worker)** function in the smart contract. Finally, the transaction data is stored in the blockchain. Then, the agent will execute the second action, such as “done”. The agent will stop action and return a word, such as “The program has been completed.....”.

IV. SYSTEM IMPLEMENTATION

A. PLATFORM AND SETUP

This paper uses Python programming language to develop the decentralized autonomous collaboration framework (DeCo-Agent) based on Ganache, Remix tools, MetaMask, and OpenAI API. Specifically, this paper uses Ganache to create a virtual Ethereum blockchain environment as the infrastructure of the platform. Ganache is a software tool widely used to create a private Ethereum blockchain environment that allows developers to simulate the Ethereum blockchain for testing and deploying smart contracts. This paper uses the Solidity programming language to write the smart contract on the Remix platform, which is an open-source Ethereum IDE for writing, compiling and debugging smart contracts. Solidity is an object-oriented programming language that is used in various blockchain platforms to implement smart contracts [33]. This paper uses MetaMask to serve as a blockchain wallet to pay for gas incurred in transactions since each transaction will consume gas in Ethereum. The blockchain wallet is a cryptocurrency wallet that allows users to manage different types of cryptocurrencies. The OpenAI API serves as LLMs in this paper, which is used to understand the input text (prompts) and generate the corresponding text to interact with smart contracts.

The Remix platform connects to the MetaMask, and MetaMask connects to Ganache, forming a full blockchain test platform. Furthermore, it imports the OpenAI API library in Python to integrate LLMs into this platform. Then, it uses Python to connect to the Ganache network, thus connecting all the components in the platform. Therefore, this paper uses these technologies and tools to form a decentralized autonomous collaboration framework.

B. LLMs PROMPTS

The prompt is critical for LLMs, and it affects the quality of the generated text [34]. This paper uses few-shot prompt engineering to build the best prompt for LLMs and return a strict response structure. The zero-shot prompting, and one-shot prompting did not work well based on the tests, and chain-of-thought prompting will be tested in the future. OpenAI mainly provides two major categories of the model endpoints (APIs): (a) *Completion* and (b) *ChatCompletion*. The *Completion* is used for a single prompt, taking a single string as input to generate text; it is suitable for older models, such as the ‘Davinci’ model. While *ChatCompletion* is used for a given dialogue and requires input in a specific format corresponding to the message history, it is particularly suitable for newer chat models, such as the ‘gpt-4’ model. The *ChatCompletion* is a higher-level API that connects the message history with the latest message from the “user”, forming all the messages into a JSON format for input. In *ChatCompletion*, the prompt can contain different roles (System, Assistant, User), which is helpful in a certain context. This paper tested two APIs and found that the *ChatCompletion* endpoint model works better for this platform. For the *ChatCompletion* endpoint

model, this paper used ‘model = gpt-4, max_tokens = 256, temperature = 0’. Therefore, this paper designed prompts are shown in List 1~List 4, which can be found in Appendix A.

Examples of prompt and response for workflow generation are shown in List 1 (Appendix A). This paper designs the prompts based on some experience from SayCan [35] which uses LLMs to reason the workflows for robot control. For instance, providing explicit numbers between steps to replace the “and then” or other phrases can instruct the LLMs to produce responses in similar patterns. Therefore, using regular expressions to process responses from OpenAI and extract each action in the subsequent is also convenient. In the sequence of action, the last action always is “done”, which design aim is used to terminate the action that denotes that all the actions have finished. Otherwise, the agent cannot stop. Furthermore, if the number of examples is less than 2, it cannot generate a precise response. It is worth noting that another key tip for designing prompts is to place parameters inside “{}”, which is important in the *Function calling*. ‘*Function calling*’, a feature provided by OpenAI, executes functions in the smart contract by calling them with parameters. In the example of List 1, LLMs will generate a strict workflow for agents based on the prompts. Therefore, users can give any query about this platform, which will generate a workflow for the agent.

The example prompt and response for action-function mapping are shown in List 2 (Appendix A). Since each action extracted from the workflow contains the word “call”. Therefore, when designing function calling prompts, there is no need to provide additional descriptions of the calling functions. The system will automatically recognize that is the calling function and parse the parameters in the action. The “tools” in the code are defined in the available actions of agents, which are used to map the action to the function of the smart contract.

The example of prompt and response for the *worker-driven* method to mapping worker-task is provided in List 3 (Appendix A), while the example of prompt and response for *task owner-driven* method to mapping task-worker is provided in List 4 (Appendix A). The “system” is the overall guideline for LLMs and is used to set the initial conditions for the dialogue, guide the behaviour of the model, or provide other contextual information. The “assistant” is the response from LLMs, robots or agents. The “user” is the end user or user agent in the conversation, which uses prompts to interact with LLMs. Following these guidelines, this paper has designed prompts like this.

C. EVALUATION

This paper employs the few-shot prompting technique to design prompts that elicit optimal responses from LLMs for DeCoAgent. Therefore, careful planning of the number of examples input to LLMs is crucial. The number of example prompts will affect the quality of LLMs generated text. Generally speaking, more example prompts will generate

TABLE 1. The correct rate of number examples.

Number Examples	Correct rate
0	0%
1	45.78%
2	100%
3	100%
4	100%
Full Prompt (10)	100%

better text but increase computational costs, such as OpenAI charges by token quantity. Using the fewest example prompts to generate more accurate responses is crucial. Therefore, this paper tests different numbers of example prompt to generate the correct response and find a suitable number of examples prompts.

This paper designs 10 question-and-answer pairs to determine the optimal number of examples to input into LLMs. Each test is independently run 100 times. Specifically, when no examples are input into the LLMs and each of the 10 questions is tested individually, the program is run 100 times for each question, and the results are recorded in a text file. When one example (a question and its corresponding answer) is input into the LLMs, and the remaining 9 questions are tested, the program is again run 100 times for each question, with the results documented. This process is repeated by incrementally increasing the number of examples input into the LLMs—from 2 examples tested against 8 questions to 10 examples tested against all 10 questions. Each scenario is run 100 times, with the results recorded accordingly. The results show that when no examples are input into the LLMs, the responses contain no correct answers. When two or more examples are input, the LLMs consistently produce correct answers. However, when only one example is input, the number of correct answers varies depending on the question. Therefore, the paper calculates the average number of correct responses to determine the accuracy rate for the one-example test. The test result is shown in Table 1, which shows that when given more than one example, LLMs will generate the correct responses to DeCoAgent.

Furthermore, this paper employs the pass@k metric [36] to evaluate the correctness of responses generated by LLMs when one example is input. The pass@k metric is designed to evaluate the functional correctness of generated code examples, addressing the limitations of traditional text similarity metrics. Specifically, pass@k is defined as the probability that at least one of the top k generated code examples for a given problem passes the unit test. Consequently, this metric is widely used in LLM research to evaluate model performance in generating correct code examples, such as [6], [37], [38], and [39]. The calculation formula of the pass@k metric

is as follows:

$$\text{pass}@k = E_{\text{problems}} \left[1 - \frac{C(n - c, k)}{C(n, k)} \right] \quad (1)$$

where, E denotes the expected value of the problem, n represents the total number of samples, c denotes the number of correct samples, and k represents the number of top samples considered. In this paper, we use $n = 100$, and k is from 1 to 100. The c is selected from the above test results. The result of distribution of different k values with corresponding the average pass@k value is show in Figure 7.

Figure 7 illustrates the pass@k value for four different scenarios: 0 example, 1 example, 2 examples and 3 examples input into the LLMs. Each scenario evaluates the average pass@k across different k values. In the 0-example scenario, the average pass@k value remains at 0, regardless of the value of k . This suggests that when no examples are provided to the LLMs as input, the LLMs fail to produce any correct response to DeCoAgent. In the 1-example, the average pass@k value asymptotically approaches a value close to 89.89% as k increases. This indicates that when only one example is provided, the LLMs cannot ensure a correct response to DeCoAgent every time. In the 2-examples and 3-examples scenario, the two curves completely overlap, and the average pass@k value remains at 1, regardless of the value of k . This indicates that when two or more examples are input into the LLMs, they consistently generate correct responses to DeCoAgent, regardless of the value of k . Furthermore, these overlapping curves suggest that additional examples do not improve the results any further. Based on these findings, the paper chooses to use two examples as prompts in LLMs for generating responses to DeCoAgent, as this is sufficient to achieve optimal accuracy.

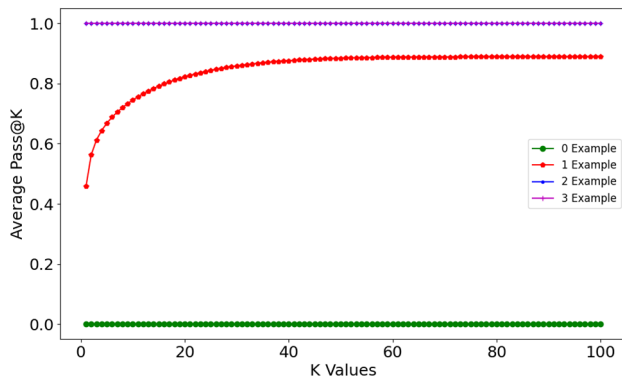


FIGURE 7. Distribution of different k values with corresponding the average pass@k value.

D. CASE STUDIES

This paper tests all the functions of the smart contract. To be consistent with the aforementioned example, this paper uses the deployment task to other agents as a case study. In this case study, the task id is changed to 6. The requirement for task ID 6 is “I have a Python bug that needs fixing”.

The example of prompt in this case studies can be seen in List 5 in Appendix A.

The LLMs respond and return “1. call Deploy Task Function with ID is {6}, 2. done.” to agents. Then agents extract the action “call Deploy Task Function with ID is {6}” sent into LLMs again. The LLMs will call the *DeployTaskFunction* function that maps the *deployTask(6)* function and execute it in the smart contract. In the execution of this function process, agents will automatically call the *owner_search_worker(6)* function to match a suitable worker and return the address of the worker for deploying this task to the worker. The LLMs return a worker address whose capability is “I am a programmer”. When the action is “done”, the agent stops all the action. All the actions are finished, which seems very good.

However, other agents pose this similar capability, such as “I am a Python programmer”, “I am a programmer expert”, and “I am a Java programmer” in the system. Therefore, the results do seem not a good answer. The “I am a Python programmer” may be a better answer. To find a better answer, this paper tests this sample many times. LLMs sometimes return “I am a Python programmer”, “I am a programmer”, and “I am a programmer expert”. The answer is not constant. When faced with such a situation, all results should be returned to the user and let the user make a decision. This may be a good method. Therefore, LLMs still need to be improved.

In conclusion, the implementation results show LLMs can work well in the system and assist the user in performing tasks and interacting with smart contracts, which indicates that the decentralized autonomous collaboration framework proposed in this paper is feasible.

V. RELATED WORK

The most relevant work involves using LLMs to solve agent collaboration problems. Some papers are focused on agent collaboration. For example, Ouyang et.al proposed an AI collaboration framework based on blockchain and smart contracts (learning markets, LM) to address collaboration models that are no longer applicable due to large-scale distributed collaboration [40]. Harris et al. proposed a decentralized and collaborative framework based on blockchain for the participant to collaboratively build a dataset to continuously update machine learning models [41]. Zhang et al. proposed Collaborative Q-learning (CollaQ) to address the multi-agent reinforcement learning issues that require orders-of-magnitude more training rounds, using joint optimization on reward assignment for multi-agent collaboration [42]. However, these methods essentially involve collaboration in sharing data, models, and computing resources, which do not address the collaboration of AI agents.

In addition, agent collaboration based on LLMs has been proposed recently. For instance, Chen et al. proposed a multi-agent framework (AgentVerse) that can orchestrate a collaborative group of agents as a system to improve the efficiency and effectiveness of task accomplishment, inspired by human group dynamics [43]. Hong et al. proposed an

innovative meta-programming framework (MetaGPT) to solve complex tasks with logic inconsistencies. MetaGPT incorporates efficient human workflows into LLM-based multi-agent collaboration, encoding standardized operating procedures as sequences of prompts to enable a more streamlined workflow [4]. Du et al. propose that multiple LLMs debate their individual responses and reasoning processes to collaborate, improve LLM outputs, and reach a common final answer [5]. Liu et al. propose a dynamic framework (DyLAN) for collaboration on complex tasks, addressing the limitations of existing approaches where a fixed set of agents interact within a static architecture [7]. Zhao et al. propose a novel multi-robot collaboration framework (RoCo) that leverages the power of LLMs, enabling robots empowered by LLMs to discuss and collectively reason about task strategies [8]. Zhang et al. presented an adaptable framework (ProAgent) to enable LLM-based agents to effectively collaborate with other AI agents or humans to solve the challenge of cooperating with unfamiliar teammates [29]. Wu et al. developed an open-source framework (AutoGen) to enable developers to develop LLM applications through the collaboration of multiple agents and these agents can converse with each other to complete tasks [6]. Hu et al. build a novel framework (Agent-CoDriver) to enable vehicles to communicate with other vehicles to achieve collaborative driving decisions [28]. These studies have a limitation in that they mainly focus on specific and limited tasks in cooperating with other agents or humans based on LLM. Alternatively, they are static approaches to agent collaboration and are based on predefined procedures. However, using smart contracts for agent collaboration in edge computing is not in their scope. This paper employs LLMs to enhance human-agent and agent-agent collaboration in edge computing through smart contracts.

VI. CONCLUSION

Focusing on addressing the assumption that all participants know each other in a local closed environment and will act with integrity for a multi-agent collaboration, this paper proposes a decentralized autonomous collaboration framework (DeCoAgent) for LLMs-empowered agents to collaborate and enable flexible and dynamic interaction. The DeCoAgent, empowered by LLMs, can understand the human natural language and execute the corresponding functions of smart contracts. This framework enables decentralized autonomous collaboration between LLM agents, allowing them to register themselves, discover the capabilities of other agents, and assign tasks on the platform. In this framework, DeCoAgent can use API interfaces to collaborate with other agents in a decentralized, open, and trustless environment, allowing users (both humans and agents) to interact with agents using natural language. The DeCoAgent framework can be applied in various application scenarios and existing agents, e.g., supply chain management, manufacturing, crowdsourcing, and complementing other existing multi-agent collaborations. The framework has been implemented on a local Ethereum blockchain, demonstrating the practical feasibility

of the proposed approach. The future work will focus on extending the platform with more functionalities, e.g. implementing worker rating to support the reputation-based worker selection, designing a confirmed mechanism to protect property when involved in payment operation in interaction,

```
Messages = [
  {'role': 'system', 'content': 'I am an AI agent
operating in the collaboration platform which runs
a smart contract on Ethereum. This smart contract
offers a wide array of functions, each serving a
specific purpose. Agent Registered Function allows
agents to become a user in smart contracts.....'},
  {'role': 'assistant', 'content': 'You can ask me
to do various tasks and I will tell you the
sequence of actions I would do to accomplish your
task.'},
  {'role': 'user', 'content': 'How would you join the
smart contract by input capability is {I am a
teacher} and contact is {test001@gmail.com}?'},
  {'role': 'assistant', 'content': '1. call Agent
Registered Function with capability is {I am a
teacher} and contact is {test001@gmail.com}, 2.
done.'},
  {'role': 'user', 'content': 'How would you post a
task in smart contract by input requirement is {I
have a question} and price is {10}?'},
  {'role': 'assistant', 'content': '1. call Post Task
Function with requirement is {I have a question}
and price is {10}, 2. done.'}
]

Response = {
  "index": 0,
  "message": {
    "role": "assistant",
    "content": "1. Call the Post Task Function
with requirement is {I have a question} and price
is {10}, 2. Done."
  },
  "logprobs": {"content": [{"token":
"1", "logprob": -0.0076062703, "bytes": [49], .....},
  "finish_reason": "stop"
}
```

LIST 1. Example of prompt and response for workflow generation.

```
Messages = [{"role": "user", "content": f"How to
{contents} in system"}]

Response = {
  "role": "assistant",
  "content": null,
  "tool_calls": [
    {
      "id": "call_afRiGHVjub6dKm4ftGIogqHK",
      "type": "function",
      "function": {
        "name": "PostTaskFunction",
        "arguments": '{"requirement": "I have a
question", "price": 10}'
      }
    }
  ]
}
```

LIST 2. Example of prompt and response for action-function mapping.

addressing the scalability of the platform, and analyzing the security and privacy protection mechanisms of the system.

APPENDIX A EXAMPLE OF PROMPT

This section lists some examples of prompts when designing the prompts in LLMs to obtain a correct response. These are shown in List 1~5.

```
Messages = [
{'role': 'system', 'content': 'I am a smart agent
and I can connect the smart contract with an agent
and do some interaction to help you do some tasks
on Ethereum.'},
{'role': 'system', 'content': 'When I give you a
user capability, you should help me find a suitable
task for me.'},
{'role': 'user', 'content': f"This is my capability
{capability}. This is task list {tasks}. Which task
do you think I'm suitable for? And tell me which
task is"}
]

Response = {
  "index": 0,
  "message": {
    "role": "assistant",
    "content": "Based on your capability as a
history teacher, the most suitable task for you
would be 'I have a history question'."
  },
  "logprobs": null,
  "finish_reason": "stop"
}
```

LIST 3. Example of prompt and response for worker driven worker- task mapping.

```
Messages = [
{'role': 'system', 'content': 'I am a smart agent
and I can connect the smart contract with an agent
and do some interaction to help you do some tasks
on Ethereum.'},
{'role': 'system', 'content': 'When I give you a
task, you should help me find a suitable worker to
do my task.'},
{'role': 'user', 'content': f"This is my task
requirement {requirement}. This is worker list
{workers}. Which worker is suitable to do my task?
And tell me who the worker is"}
]

Response = {
  "index": 0,
  "message": {
    "role": "assistant",
    "content": "Based on your task
requirement, the suitable worker for your task
would be 'I am an history teacher'."
  },
  "logprobs": null,
  "finish_reason": "stop"
}
```

LIST 4. Example of prompt and response for task owner driven task-worker mapping.

```
Messages = [
{'role': 'user', 'content': 'How would you join the
smart contract by input capability is {I am a
teacher} and contact is {test001@gmail.com}?'},
{'role': 'assistant', 'content': '1. call Agent
Registered Function with capability is {I am a
teacher} and contact is {test001@gmail.com}, 2.
done.'},
{'role': 'user', 'content': ' How would you deploy
a task to other user in smart contract by input ID
is {6}?'}
]
```

LIST 5. Example of prompt.

REFERENCES

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," 2022, *arXiv:2210.03629*.
- [2] S. Yao, D. Yu, J. Zhao, I. Shafraan, T. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2024, pp. 1–14.
- [3] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, "Large language model based multi-agents: A survey of progress and challenges," 2024, *arXiv:2402.01680*.
- [4] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. Ka Shing Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework," 2023, *arXiv:2308.00352*.
- [5] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving factuality and reasoning in language models through multiagent debate," 2023, *arXiv:2305.14325*.
- [6] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. Hassan Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," 2023, *arXiv:2308.08155*.
- [7] Z. Liu, Y. Zhang, P. Li, Y. Liu, and D. Yang, "Dynamic LLM-agent network: An LLM-agent collaboration framework with agent team optimization," 2023, *arXiv:2310.02170*.
- [8] Z. Mandi, S. Jain, and S. Song, "RoCo: Dialectic multi-robot collaboration with large language models," in *Proc. IEEE Int. Conf. Robot. Automat. (ICRA)*, May 2024, pp. 286–299.
- [9] J. Fu and Y. Fu, "An adaptive multi-agent system for cost collaborative management in supply chains," *Eng. Appl. Artif. Intell.*, vol. 44, pp. 91–100, Sep. 2015.
- [10] J. Zhang and X. Wang, "Multi-agent-based hierarchical collaborative scheduling in re-entrant manufacturing systems," *Int. J. Prod. Res.*, vol. 54, no. 23, pp. 7043–7059, Dec. 2016.
- [11] H. Rahman, S. B. Roy, S. Thirumuruganathan, S. Amer-Yahia, and G. Das, "Task assignment optimization in collaborative crowdsourcing," in *Proc. IEEE Int. Conf. Data Mining*, Nov. 2015, pp. 949–954.
- [12] E. Kasneci, K. Seßler, S. Küchemann, M. Bannert, D. Dementieva, F. Fischer, U. Gasser, and S. Günnemann, "ChatGPT for good? On opportunities and challenges of large language models for education," *Learn. Individual Differences*, vol. 103, Apr. 2023, Art. no. 102274.
- [13] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, no. 1, pp. 5485–5551, 2020.
- [14] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.
- [15] R. Thoppilan et al., "LaMDA: Language models for dialog applications," 2022, *arXiv:2201.08239*.
- [16] J. W. Rae et al., "Scaling language models: Methods, analysis & insights from training gopher," 2021, *arXiv:2112.11446*.
- [17] A. Chowdhery et al., "PaLM: Scaling language modeling with pathways," 2022, *arXiv:2204.02311*.
- [18] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. Victoria Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, P. Singh Koura, A. Sridhar, T. Wang, and L. Zettlemoyer, "OPT: Open pre-trained transformer language models," 2022, *arXiv:2205.01068*.

- [19] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, “LLaMA: Open and efficient foundation language models,” 2023, *arXiv:2302.13971*.
- [20] Y. Sun et al., “ERNIE 3.0: Large-scale knowledge enhanced pre-training for language understanding and generation,” 2021, *arXiv:2107.02137*.
- [21] H. Li, “Language models: Past, present, and future,” *Commun. ACM*, vol. 65, no. 7, pp. 56–63, 2022.
- [22] S. Arvidsson and J. Axell, “Prompt engineering guidelines for LLMs in requirements engineering,” M.S. thesis, Univ. Gothenburg, Chalmers Univ. Technol., 2023.
- [23] B. Meskó, “Prompt engineering as an important emerging skill for medical professionals: Tutorial,” *J. Med. Internet Res.*, vol. 25, Oct. 2023, Art. no. e50638.
- [24] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. Wei Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, “Finetuned language models are zero-shot learners,” 2021, *arXiv:2109.01652*.
- [25] D. N. M. Hoang, M. Cho, T. Merth, M. Rastegari, and Z. Wang, “Do compressed LLMs forget knowledge? An experimental study with practical implications,” 2023, *arXiv:2310.00867*.
- [26] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 24824–24837.
- [27] L. Wang, J. Zhang, H. Yang, Z. Chen, J. Tang, Z. Zhang, X. Chen, Y. Lin, R. Song, W. Xin Zhao, J. Xu, Z. Dou, J. Wang, and J.-R. Wen, “User behavior simulation with large language model based agents,” 2023, *arXiv:2306.02552*.
- [28] S. Hu, Z. Fang, Z. Fang, Y. Deng, X. Chen, and Y. Fang, “AgentsCoDriver: Large language model empowered collaborative driving with lifelong learning,” 2024, *arXiv:2404.06345*.
- [29] C. Zhang, K. Yang, S. Hu, Z. Wang, G. Li, Y. Sun, and Y. Yang, “ProAgent: Building proactive cooperative agents with large language models,” in *Proc. AAAI Conf. Artif. Intell.*, Mar. 2024, vol. 38, no. 16, pp. 17591–17599.
- [30] S. Han, Q. Zhang, Y. Yao, W. Jin, Z. Xu, and C. He, “LLM multi-agent systems: Challenges and open problems,” 2024, *arXiv:2402.03578*.
- [31] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, “A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions,” 2023, *arXiv:2311.05232*.
- [32] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 9459–9474.
- [33] C.-C. Hung, K. Chen, and C.-F. Liao, “Modularizing cross-cutting concerns with aspect-oriented extensions for solidity,” in *Proc. IEEE Int. Conf. Decentralized Appl. Infrastructures (DAPPCON)*, Apr. 2019, pp. 176–181.
- [34] H. Ratnayake and C. Wang, “A prompting framework to enhance language model output,” in *Proc. Australas. Joint Conf. Artif. Intell.*, Singapore: Springer, 2023, pp. 66–81.
- [35] M. Ahn et al., “Do as I can, not as I say: Grounding language in robotic affordances,” 2022, *arXiv:2204.01691*.
- [36] M. Chen et al., “Evaluating large language models trained on code,” 2021, *arXiv:2107.03374*.
- [37] Z.-C. Lyu, X.-Y. Li, Z. Xie, and M. Li, “Top pass: Improve code generation by pass@k-maximized code ranking,” 2024, *arXiv:2408.05715*.
- [38] P. Wu, N. Guo, X. Xiao, W. Li, X. Ye, and D. Fan, “ITERTL: An iterative framework for fine-tuning LLMs for RTL code generation,” 2024, *arXiv:2407.12022*.
- [39] R. Qiu, W. Will Zeng, H. Tong, J. Ezick, and C. Lott, “How efficient is LLM-generated code? A rigorous & high-standard benchmark,” 2024, *arXiv:2406.06647*.
- [40] L. Ouyang, Y. Yuan, and F.-Y. Wang, “Learning markets: An AI collaboration framework based on blockchain and smart contracts,” *IEEE Internet Things J.*, vol. 9, no. 16, pp. 14273–14286, Aug. 2022.
- [41] J. D. Harris and B. Waggoner, “Decentralized and collaborative AI on blockchain,” in *Proc. IEEE Int. Conf. Blockchain (Blockchain)*, Jul. 2019, pp. 368–375.
- [42] T. Zhang, H. Xu, X. Wang, Y. Wu, K. Keutzer, J. E. Gonzalez, and Y. Tian, “Multi-agent collaboration via reward attribution decomposition,” 2020, *arXiv:2010.08531*.

- [43] W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C.-M. Chan, H. Yu, Y. Lu, Y.-H. Hung, C. Qian, Y. Qin, X. Cong, R. Xie, Z. Liu, M. Sun, and J. Zhou, “AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors,” in *Proc. 12th Int. Conf. Learn. Represent.*, 2023, pp. 1–43.



ANAN JIN received the B.Eng. degree in software engineering and the M.S. degree in computer science and technology from the East China University of Technology (ECUT), China, in 2018 and 2021, respectively. He is currently pursuing the Ph.D. degree with the Technological University of the Shannon: Midlands Midwest (TUS). His research interests include blockchain technology, edge computing, and distributed systems.



YUHAN YE received the B.Eng. and M.Eng.Sc. degrees in electronic engineering from the National University of Ireland, Maynooth, Ireland, and the Ph.D. degree from Athlone Institute of Technology, Ireland. He is currently an Assistant Lecturer with the Department of Computer and Software Engineering, Technological University of the Shannon: Midlands Midwest. His current research interests include computer networking, adversarial machine learning, and multimedia communication.



BRIAN LEE (Member, IEEE) received the Ph.D. degree in computer science from Trinity College Dublin, in 2004. He is the Director of the Software Research Institute (SRI), Technological University of the Shannon: Midlands Midwest, Ireland. He is a Science Foundation Ireland (SFI) Funded Investigator with the SFI CONFIRM Smart Manufacturing Centre. His research interests include computer security (access control, network security, and security analytics), programmable networking, and edge computing. He is a member of IEEE (Communications, Computer and Robotics, and Automation societies) and ACM.



YUANSONG QIAO (Member, IEEE) received the Ph.D. degree in computer applied technology from the Institute of Software, Chinese Academy of Sciences, Beijing, China, in 2008. He is a Senior Research Fellow with the Software Research Institute (SRI), Technological University of the Shannon: Midlands Midwest, Ireland. He is a Science Foundation Ireland (SFI) Funded Investigator with the SFI CONFIRM Smart Manufacturing Centre. His research interests include future internet architecture, blockchain systems, robotic control and coordination, and edge intelligence and computing. He is a member of IEEE (Communications, Computer and Robotics and Automation societies and Blockchain Community) and ACM (SIGCOMM and SIGMM).

...